

Gold table layout — the decision rule

Why attributes + financials merge into one table, while clickstream stays apart

Decision rule

Tables stay separate when update cadence, ownership, grain, or reliability profile differ. They merge only when all four align AND they are nearly always consumed together.

Compatibility matrix across the three feature sources

Dimension	Attributes	Financials	Clickstream
Update cadence	Monthly batch	Monthly batch	Hourly stream
System of record	Core banking	Core banking	Web analytics
Grain	Customer × month	Customer × month	Customer × month
Population coverage	All customers	All customers	Web-active only
Schema volatility	Low	Low	High (new events)
Verdict vs attributes	—	match	differ

Outcome

MERGE → feature_store_customer

attributes ⋈ financials. Four dimensions align; analysts always use them together.

SPLIT → feature_store_clickstream

Cadence, owner, and coverage all differ. Joining at Gold would inject staleness or silently drop rows.

Business framing: the split mirrors source-system boundaries — one table per (system of record × refresh cadence).

08

Why feature store $\neq$ label store

Decoupling features from outcomes protects against leakage and unblocks refresh cadence

Different time semantics

Features are observed AT loan application time. Labels are observed 6 months LATER. Mixed in one table, any join on snapshot_date silently creates target leakage.

Different refresh triggers

Features refresh as upstream snapshots land. Labels can't crystallise until mob=6 is reached. Coupling them means feature refresh would be blocked waiting for label maturity.

Different consumers

The same feature store can feed multiple models — default at mob=3, churn, prepayment, fraud. Decoupling makes the feature store a reusable asset, not a one-model artefact.

Different audit story

Regulators ask: 'what did the model know at decision time?' That question has a clean answer only when features and outcomes live in separate tables with separate timestamps.

ML training time — leakage is forced into the open

`feature_store` $\bowtie$ `label_store` ON `Customer_ID, snapshot_date` → an explicit, reviewable join. No leak can sneak in by accident.